

07. 트랜스포머와 전이 학습

어텐션이 실제로 무슨 계산인지 손으로 따라가 보고, 그 위에 세워진 BERT와 전이학습으로 넘어간다.

1. 어텐션의 직관 — 가중합

거창한 개념이 아니라 **가중 평균**이다. 문장에서 어떤 단어를 얼마나 볼지 점수를 매기고, 그 비율대로 섞는다.

```
sentence = ["The", "cat", "sat", "on", "the", "mat"]
attention_scores = {"The": 0.1, "cat": 1.0, "sat": 0.8,
                    "on": 0.3, "the": 0.1, "mat": 0.6}
```

cat 을 기준으로 새 벡터를 만든다면, 각 단어 벡터에 점수를 곱해 더한다.

```
for dim in range(3):
    weighted_sum = (0.1 * word_vectors["The"][dim] +
                    1.0 * word_vectors["cat"][dim] +
                    0.8 * word_vectors["sat"][dim] + ...)
```

값을 추적해 보면:

	벡터
원래 cat	[0.5, 0.8, 0.2]
어텐션 적용 후	[1.62, 1.28, 1.37]

cat 혼자만의 표현이 아니라 **문맥이 섞인 표현**으로 바뀌었다. 이게 어텐션이 하는 일의 전부다.

2. Q, K, V — 세 가지 역할

같은 임베딩에서 세 개의 다른 행렬을 곱해 세 가지 역할을 만든다.

```
Q = embeddings @ W_q    # Query   : 내가 찾는 것
K = embeddings @ W_k    # Key     : 내가 가진 것
V = embeddings @ W_v    # Value   : 실제로 전달할 내용
```

이름	비유	역할
Query	검색어	이 단어가 무엇을 궁금해하는가
Key	색인	이 단어가 어떤 정보를 갖고 있는가
Value	내용	선택되면 실제로 넘길 값

Scaled Dot-Product Attention

```
scores = Q @ K.T / np.sqrt(self.d_model)
attention_weights = F.softmax(scores, dim = 1)
output = attention_weights @ V
```

왜 sqrt(d_model) 로 나누는가 — 차원이 커질수록 내적 값의 분산이 커진다. d_model = 512 면 점수가 수십~수백까지 튀는데, 그 상태로 softmax를 통과하면 한 곳에 1, 나머지는 0으로 쏠려 기울기가 거의 0이 된다. 나눠서 분산을 안정시키는 장치다.

dim = 1 은 **각 행(단어)의 가중치 합이 1**이 되게 한다. 축을 잘못 잡으면 열 방향으로 정규화되어 의미가 달라진다.

shape 흐름

단계	shape (단어 6개, d_model 512 기준)
embeddings	(6, 512)
Q, K, V	(6, 512)
scores = Q @ K.T	(6, 6)
attention_weights	(6, 6)
output = weights @ V	(6, 512)

(6, 6) 행렬이 "어느 단어가 어느 단어를 보는가"의 지도다. 이걸 그대로 히트맵으로 그리면 모델의 판단 근거가 보인다.

3. 어텐션 시각화

```
plt.imshow(attention_matrix, cmap = "Blues")
plt.xticks(range(len(words)), words, rotation = 45)
plt.yticks(range(len(words)), words)
```

읽는 법은 **행 기준**이다. **i** 번째 행은 "단어 i가 다른 단어들을 얼마나 보는가"를 뜻한다.

	AI	learns	to	understand	language
AI	0.9	0.1	0.0	0.0	0.0
learns	0.2	0.7	0.1	0.0	0.0
to	0.0	0.3	0.6	0.1	0.0
understand	0.1	0.0	0.1	0.7	0.1
language	0.0	0.0	0.0	0.4	0.6

대각선이 진하면 자기 자신을 주로 본다는 뜻이고, language 행이 understand 를 0.4로 보는 건 두 단어가 의미적으로 묶였다는 신호다.

시각화 시 `.detach().numpy()` 처럼 **괄호를 꼭 붙여야** 한다. 괄호를 빼면 메서드 객체가 넘어가 `imshow` 에서 터진다.

4. 멀티헤드 어텐션

```
self.multihead_attn = nn.MultiheadAttention(embed_dim = 512,
                                             num_heads = 8,
                                             batch_first = True)

attn_output, attn_weights = self.multihead_attn(x, x, x)
```

Q, K, V에 **같은 입력을 세 번** 넣는 게 Self-Attention이다. 문장이 자기 자신을 본다는 뜻이다.

헤드를 8개로 나누는 이유는 관점을 나누기 위해서다. 512차원을 64차원씩 8조각으로 쪼개 각각 다른 관계를 학습하게 한다. 어떤 헤드는 문법 관계를, 어떤 헤드는 의미 관계를 잡는 식이다.

출력	shape (batch 2, seq 10, embed 512)
attn_output	(2, 10, 512)
attn_weights	(2, 10, 10)

분류로 잇기

```
pooled = attn_output.mean(dim = 1)      # 시퀀스 축 평균 → (2, 512)
predictions = classifier(pooled)       # (2, 3)
```

시퀀스를 하나의 벡터로 요약할 때 평균 풀링을 쓴다. 마지막 시점만 쓰는 RNN과 달리, 어텐션은 이미 문맥이 섞여 있어 평균이 잘 통한다.

5. 트랜스포머 구조

인코더는 문장을 벡터로, **디코더**는 벡터를 문장으로 바꾼다.

구성 요소	역할
Multi-Head Attention	문맥 정보 섞기
Feed Forward	위치별 비선형 변환
Residual + LayerNorm	깊게 쌓아도 학습되게
Positional Encoding	순서 정보 주입

위치 인코딩이 필요한 이유 — 어텐션은 가중합이라 순서 개념이 없다. [cat, sat] 과 [sat, cat] 이 같은 결과를 낸다. 그래서 위치 정보를 임베딩에 더해준다. RNN은 구조상 순서를 타지만 트랜스포머는 따로 넣어줘야 한다.

이게 트랜스포머가 RNN보다 빠른 이유이기도 하다. 순차 처리가 없으니 **전체 시퀀스를 병렬로** 계산한다.

6. BERT 사용하기

```
from transformers import BertTokenizer, BertForSequenceClassification

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained("bert-base-uncased",
                                                    num_labels = 2)

inputs = tokenizer(sentence,
                    return_tensors = "pt",
                    truncation = True,
                    padding = True)

outputs = model(**inputs)
logits = outputs.logits
predicted_class = torch.argmax(logits, dim = 1)
```

return_tensors = "pt" 는 복수형

인자	반환 형태
return_tensors = "pt"	{"input_ids": tensor([[101, ...]]), ...}
return_tensor = "pt" (단수)	{"input_ids": [101, 2023, ...]} — 리스트

토큰나이저는 모르는 키워드를 **조용히 무시**한다. 에러가 안 나고 파이썬 리스트가 돌아와서, 모델에 넣는 순간 엉뚱한 곳에서 터진다.

****inputs 로 펼치기**

tokenizer 는 딕셔너리를 반환한다. ** 로 펼쳐 넣으면 input_ids , attention_mask , token_type_ids 가 각각 인자로 전달된다.

변수명을 input 으로 두면 파이썬 **내장 함수**와 겹친다. NameError 가 아니라 argument after ** must be a mapping, not method 같은 엉뚱한 메시지가 나와서 원인 찾기가 어렵다.

7. 미세조정 두 가지 방식

```
# 방식 1 : 특징 추출 (Feature Extraction)
model_fe = BertForSequenceClassification.from_pretrained(...)
for param in model_fe.bert.parameters():
    param.requires_grad = False      # 백본 동결

# 방식 2 : 전체 미세조정 (Fine-Tuning)
model_ft = BertForSequenceClassification.from_pretrained(...)

# 동결 없음
```

	특징 추출	전체 미세조정
학습 대상	분류 헤드만	전부
속도	빠름	느림
데이터 요구량	적어도 됨	많이 필요
성능	보통	더 높음
과적합 위험	낮음	높음

데이터가 수백 건 수준이면 동결이 낫고, 수천 건 이상이면 전체 미세조정이 보통 이긴다.

학습률은 둘 다 2e-5 근처를 쓴다. 사전학습 가중치를 크게 흔들면 안 되기 때문에 처음부터 학습할 때보다 훨씬 작게 잡는다.

8. ResNet 전이학습

```
import torchvision.models as models

resnet18 = models.resnet18(weights = models.ResNet18_Weights.DEFAULT)

num_fts = resnet18.fc.in_features
resnet18.fc = nn.Linear(num_fts, 10)      # 출력층 교체
```

부분	상태
conv 계층 (특징 추출)	ImageNet 사전학습 완료
fc (분류)	무작위 초기화

in_features 를 하드코딩하지 않고 읽어오는 게 요령이다. 백본을 resnet50 으로 바꿔도 코드가 안 깨진다.

이미지 전처리는 학습 때와 똑같이

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean = [0.485, 0.456, 0.406],
                          std  = [0.229, 0.224, 0.225])])
```

이 평균과 표준편차는 **ImageNet 통계값**이다. 사전학습 모델은 이 분포를 전제로 학습되었기 때문에, 0~1 그대로 넣으면 성능이 크게 떨어진다.

cv2.imread 를 쓰신다면 BGR로 읽히니 cv2.COLOR_BGR2RGB 변환을 거쳐야 한다. PIL은 RGB라 .convert("RGB") 면 충분하다.

pretrained 인자는 구식

```
resnet18 = models.resnet18(pretrained = True)          # 경고 발생
resnet18 = models.resnet18(weights = models.ResNet18_Weights.DEFAULT) # 권장
```

torchvision 0.13부터 weights 로 바뀌었다. 경고일 뿐 동작에는 문제가 없지만, 향후 제거될 수 있다.

검수에서 찾은 동작 오류

1. 출력이 의미 없음 — 분류 헤드가 무작위

BERT와 ResNet 예제 모두 같은 상황이다.

```
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels = 2)
resnet18.fc = nn.Linear(num_fts, 10)
```

사전학습 본체에 **학습된 적 없는 새 출력층**을 붙인 상태다. 어떤 문장이나 이미지를 넣어도 결과는 사실상 난수다. 입력을 바꿔가며 결과를 비교해도 의미가 없다.

전이학습에서 이게 정상적인 출발점이다. 실제 감성 판별을 보려면 이미 미세조정된 체크포인트를 써야 한다.

```
name = "distilbert-base-uncased-finetuned-sst-2-english"
```

이 모델은 model.config.id2label 에 {0: 'NEGATIVE', 1: 'POSITIVE'} 가 들어 있어 라벨 이름까지 확인된다.

2. attention_mask 누락

```
dataset = TensorDataset(encoded_inputs["input_ids"], labels) # mask 빠짐
```

padding = True 로 길이를 맞춰놓고 input_ids 만 데이터셋에 넣으면, 모델이 패딩 토큰([PAD])도 실제 단어로 취급한다.

문장	실제 토큰	패딩 후
Fantastic film	4	6 (패딩 2개)
I love this movie	6	6

어텐션이 의미 없는 자리까지 계산한다. 학습은 돌아가지만 결과가 왜곡된다. encoded_inputs["attention_mask"] 도 함께 넣어야 한다.

3. 배치 크기 불일치 — 단수와 복수

```
for input_ids, label in train_loader:
    outputs = model(input_ids = input_ids, labels = labels) # 잘못
    outputs = model(input_ids = input_ids, labels = label) # 정상
```

루프에서 꺼낸 건 배치 단위 label 인데, 바깥의 전체 labels 를 넘기고 있다.

변수	크기
input_ids (배치)	(2, 6)
label (배치)	(2,)
labels (전체)	(5,)

labels 가 바깥에 정의되어 있어 NameError 가 아니라 실행 시점의 크기 불일치로 터진다.

4. 대문자 k vs K

```
scores = Q @ k.T / np.sqrt(self.d_model) # 소문자 k는 정의된 적 없음
```

파이썬은 대소문자를 구분한다. 위에서 만든 건 K 다.

정리 포인트

- 어텐션은 **가중합**. 각 단어가 문맥이 섞인 표현으로 바뀐다
- sqrt(d_model) 로 나누는 건 softmax가 한쪽으로 쏠려 기울기가 죽는 걸 막기 위함
- Self-Attention은 Q, K, V에 **같은 입력**을 넣는 것
- 트랜스포머는 순서 개념이 없어 **위치 인코딩**이 필요하고, 덕분에 병렬 처리가 된다
- return_tensors 는 복수형. 단수로 쓰면 조용히 리스트가 돌아온다
- **inputs 로 펼쳐 넣기. 변수명 input 은 내장 함수와 충돌
- 동결(특징 추출) vs 전체 미세조정은 **데이터 양**으로 고른다
- 사전학습 모델에는 **ImageNet 정규화**를 그대로 적용
- 헤드를 갈아끼운 직후의 출력은 의미가 없다. 학습이 먼저